Thalamus — multi-cortex research agent

SPEC-ARCH-04 v0.2 OVERVIEW Jeremie Nunez

Entry point

`packages/thalamus/src/config/container.ts` is the composition entry point. It wires repositories, `CortexRegistry`, execution strategies, `ThalamusPlanner`, `ThalamusDAGExecutor`, reflexion, stop criteria, cycle loop, finding persister, and `ResearchGraphService`. The app owns concrete domain adapters and passes them in through `BuildThalamusOpts`: `skillsDir`, `CortexDataProvider`, `DomainConfig`, `WebSearchPort`, `EmbedderPort`, `EntityCatalogPort`, and `SourceFetcherPort`.

`ThalamusService.runCycle` is the runtime entry point. It resolves the DAG source in this order: explicit `dag`, `daemonJob`, manual `cortices`, then LLM planner. Explicit and manual DAGs are still finalized through runtime filters before execution.

Complexity checkpoint

Thalamus is the *producer* loop, not the reviewer loop. Its hard artifact is a confidence-gated knowledge graph finding, linked to the cycle through `research_cycle_finding`. The cycle can use a retrieval swarm through the SSA explorer pack, but that swarm returns source candidates; it does not create reviewable Sweep suggestions. The control path is therefore:

> `runCycle` → resolve/finalize DAG → execute cortices → reflexion/replan → confidence filter → graph persistence.

This distinction matters because the repository also contains Sweep audit swarms and Fish counterfactual swarms. They share the fan-out idea, but Thalamus owns planning, reflexion, and KG persistence; it does not own Redis review state, resolution locks, or fish memory.

1 Orchestration sequence

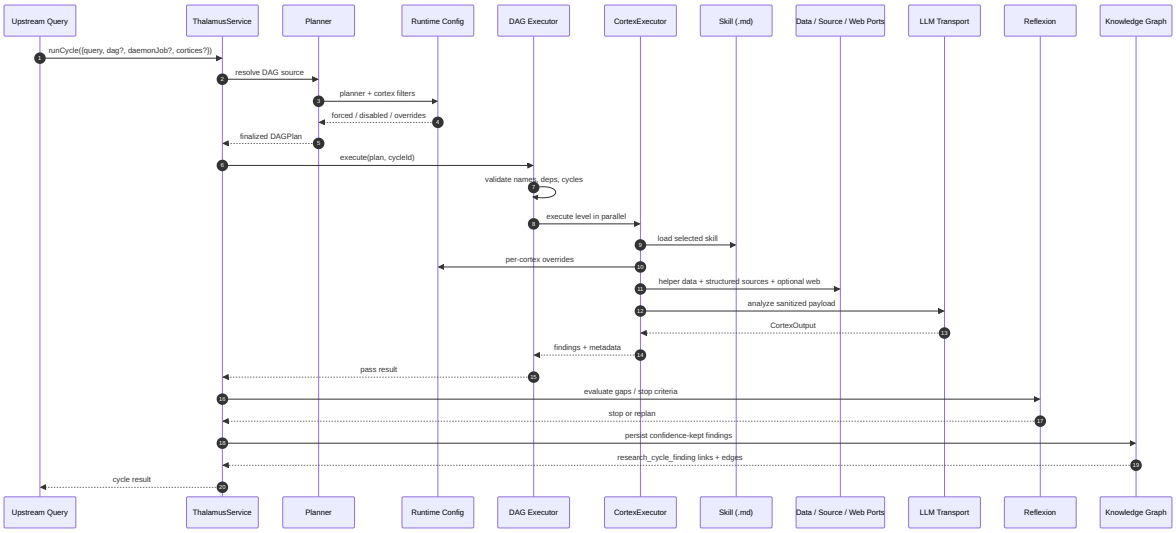

Figure 1: Cycle dispatch — plan resolution, runtime filters, DAG execution, reflexion, and knowledge-graph persistence.

2 Cortex anatomy

The current kernel does not own per-domain cortex folders. A cortex capability is a markdown skill file in the app-provided `skillsDir`. `CortexRegistry` parses frontmatter (`name`, `description`, `sqlHelper`, `params`, strategy metadata) and exposes the skill body to execution strategies. The kernel stays generic by calling named helpers through `CortexDataProvider` and structured-source enrichers through `SourceFetcherPort`.

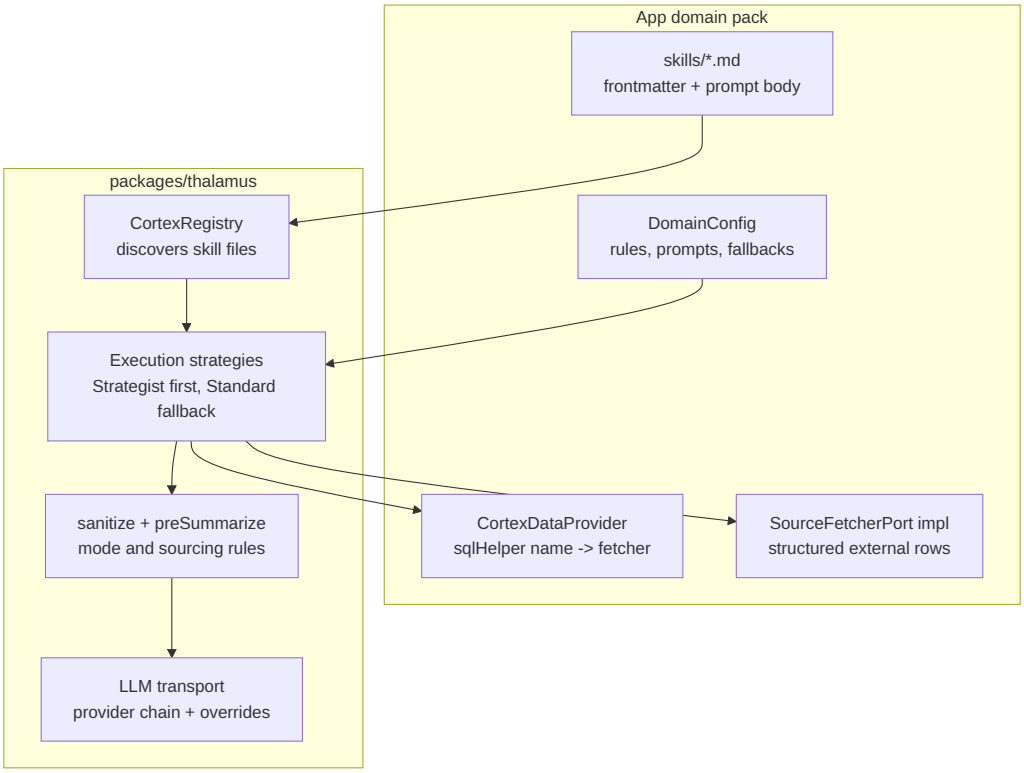

Figure 2: Cortex execution anatomy — skill files are app-owned; SQL-like helpers, structured sources, web search, and domain rules enter through ports.

`StandardStrategy` is the catch-all path: run the injected data helper, fetch structured sources, optionally call web search, pre-summarize and sanitize payloads, call the LLM, normalize findings, and emit a meta-finding when non-empty data produces no accepted finding. `StrategistStrategy`

is a synthesis path over prior findings and runs without SQL/source/web enrichment.

3 Planner and runtime filters

Planning is no longer just prompt discipline. The planner LLM emits a `DAGPlan`, then the kernel applies hard post-filters:

- unknown cortex names are dropped for LLM-produced plans and rejected by the DAG executor if they reach execution;
- `thalamus.planner.disabledCortices` and `thalamus.cortex.overrides[{name}].enabled=false` strip nodes and prune dependencies;
- `forcedCortices`, `maxCortices`, and `mandatoryStrategist` are enforced after filtering;
- user-scoped cortices are stripped when no user context exists, including daemon DAGs and explicit/manual DAGs after finalization.

Runtime config is injected through `ConfigProvider<T>` setters. The registered Thalamus runtime-config domains are nano, nano-swarm, planner, cortex, reflexion, and budgets; transport provider credentials are consumed through the separate transport config provider, not a `RuntimeConfigDomain`. Some declared fields are intentionally ahead of usage; the reliable execution knobs today are disabled/enabled cortices, forced cortices, per-cortex provider/model/timeout overrides, budget rows, `thalamus.reflexion.maxIterations`, transport mode/provider credentials, and nano-swarm wave sizing. Other reflexion fields are registered but not yet consumed by `ThalamusService`.

4 Explorer — nano swarm economics

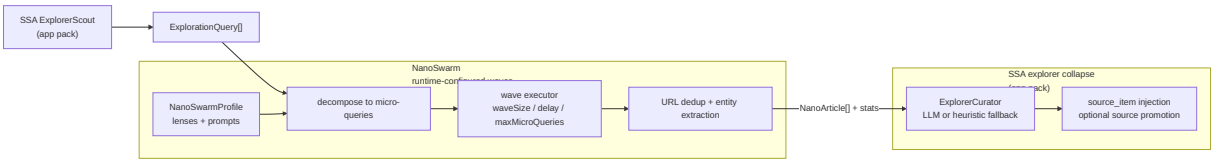

Figure 3: Nano swarm fan-out is kernel-generic; SSA scout/crawler/curator/orchestrator live in the app domain pack.

`packages/thalamus/src/explorer/` exposes only `nano-caller.ts` and `nano-swarm.ts`. The swarm decomposes scout queries using an injected `NanoSwarmProfile`, runs `callNanoWithMode`

in waves (`waveSize`, `waveDelayMs`, `maxMicroQueries`), deduplicates URLs, extracts opaque domain entities through an injected extractor, and returns generic `NanoArticle[]` plus stats. SSA's `ExplorerScout`, crawler, curator, source injection, and source promotion live under `apps/console-api/src/agen`

Why this beats a single strong model: the expensive reasoning call is reserved for judgment and curation, while retrieval fan-out is bounded by wave config and can fail partially without blocking the whole exploration pass.

5 Reflexion loop

After each DAG pass, the loop filters findings by the current confidence threshold, evaluates stop criteria, optionally asks reflexion for gaps, and replans with prior titles plus the gap list. Two identical gap rounds trigger `gap-plateau`; exhausting the configured iteration budget returns `max-iterations`. The verification heuristic is domain-gated through `DomainConfig.isVerificationRelevantEntityType`; when absent, every edge entity type remains verification-relevant.

6 Knowledge graph persistence

Findings land in Postgres through `ResearchGraphService`. The persistent surface is `research_cycle`, `research_finding`, `research_cycle_finding`, and `research_edge`. The cycle-finding junction is now the source of truth for what a cycle surfaced. Re-emitted dedup hits are linked idempotently, and the cycle's finding count increments only when a new junction row is created.

Semantic dedup uses the injected `EmbedderPort`; when it is unavailable the graph service falls back to hash / non-semantic behavior. Cross-links and entity resolution stay behind repositories and ports; the LLM never writes ad-hoc SQL.

7 Drift notes (v0.1 → v0.2)

- Replaced the removed `orchestrators/executor.ts` entry point with `buildThalamusContainer` and `ThalamusService.runCycle`.
- Corrected cortex anatomy: app-owned skill files plus injected helper/source ports, not kernel-owned `cortices/{domain}` folders.
- Clarified that explorer scout/crawler/curator/orchestrator are SSA app-pack code; Thalamus owns only the generic nano caller and wave executor.
- Added current runtime-filter behavior for explicit, daemon, manual, and LLM-generated DAGs.
- Added the junction-table counting rule and domain-gated verification heuristic introduced in the current worktree.